计算机组成与原理.

40% 考试

20% 实验

20% 项目（浮点运算）

10% 作业.

10% 考勤.

KB = 10^3 KiB = 2^{10}.

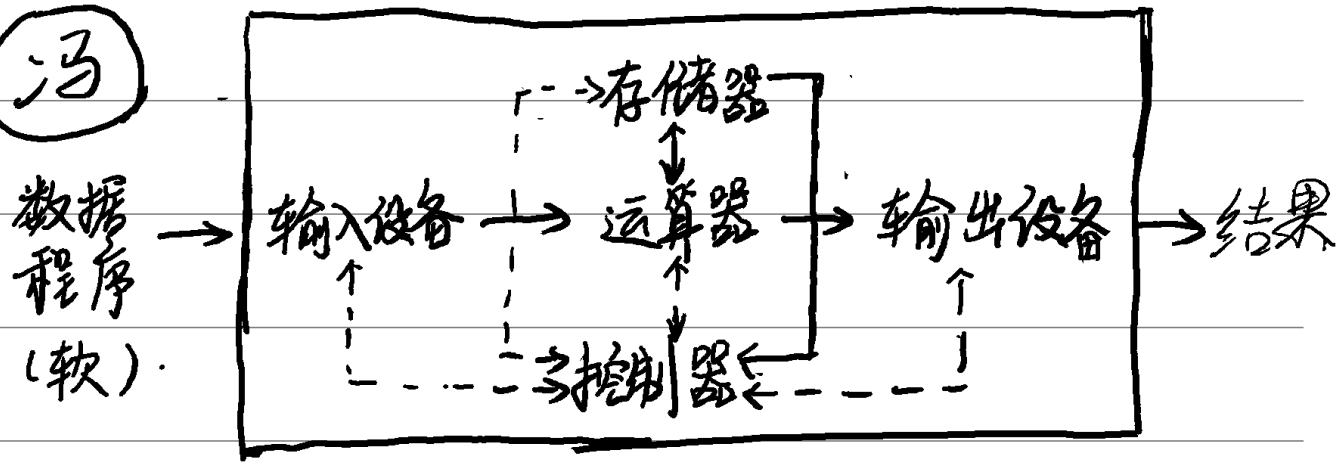
MB = 10^6 MiB = 2^{20}.

GB = 10^9 GiB = 2^{30}.

⇒ T、P、E、Z、Y

硬件基本组成

冯



输入：接收数据、指令集合.

存储器：存放数据与程序 Memory

运算器：算术/逻辑运算 PU.

输出：转化为人类熟悉形式

控制器：解析指令. CU

特点：① 五大件

② 指令、数据同地位储存

可按地址寻访

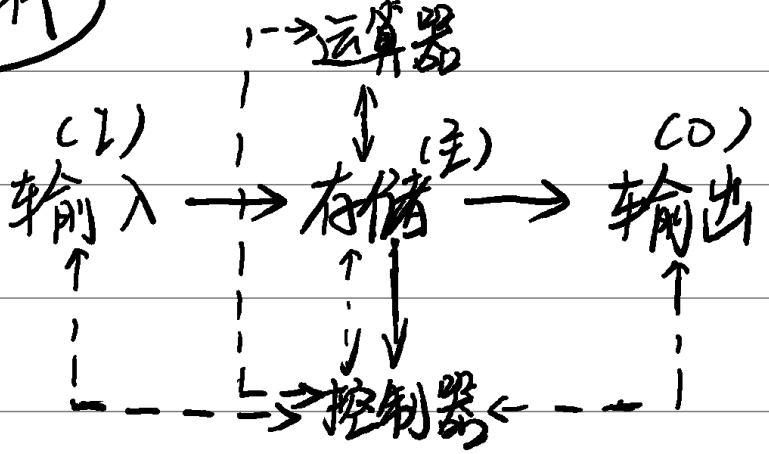
③ 指令和数据用二进制表示

④ 指令由操作码和地址码组成.

⑤ 存储程序　　操作码 → 运算　　地址码 → 内存单元

⑥ 以运算器为中心.

现代



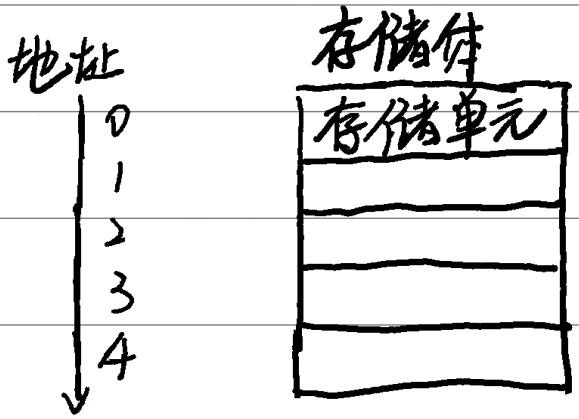
CPU = 运算器 + 控制器.

运算器 } CPU
控制器 }
存储器 { 主存 } 主机
辅存 } 硬件.

I/O设备

§ 主存储器

主存储器 { 存储体（存放数据）
MAR 存储地址寄存器
MDR 存储数据寄存器



8bit × k（字长）

存储元：
1bit 存储电子元件

MAR 位数反映存储单元个数

MDR 位数 = 存储字长.

1 B (Byte、字节) = 8 b (bit)

1 个字 (word) = 16 bit

§ 运算器

1. ACC：累加器，用于存放操作数或运算结果.

2. MQ：乘商寄存器，乘除运算时，用于存放操作数/运算结果

3. X：通用操作数寄存器，存放操作数.

4. ALU (core)：算术逻辑元，用内部电路实现算术/逻辑运算.

ACC 被加/减数，和/差 乘积高位

MQ —— 乘数&乘积低位

X 加/减数 被乘数

ACC 被除数、余数

MQ 商

X 除数

§ 控制器.

- CU：控制单元，分析指令，发出信号.
- IR：指令寄存器，存放当前指令.
- PC：程序计数器，存放下一前指令地址，有自动加1功能.

PC 取指，IR分析，CU执行.

(指令存储)

§工作过程.

(PC)地址 → MAR ---(一个微指令)

M(MAR) → MDR.

(MDR) → IR

(PC)+1 → PC 取指结束

OP(IR) → CU 分析结束

Ad(IR) → MAR.

M(MAR) → MDR.

(MDR) → ACC. 执行结束

摩尔定律：集成电路上可容纳晶体管数量，每隔18-24月翻一番，性能提升，成本降低.

软件

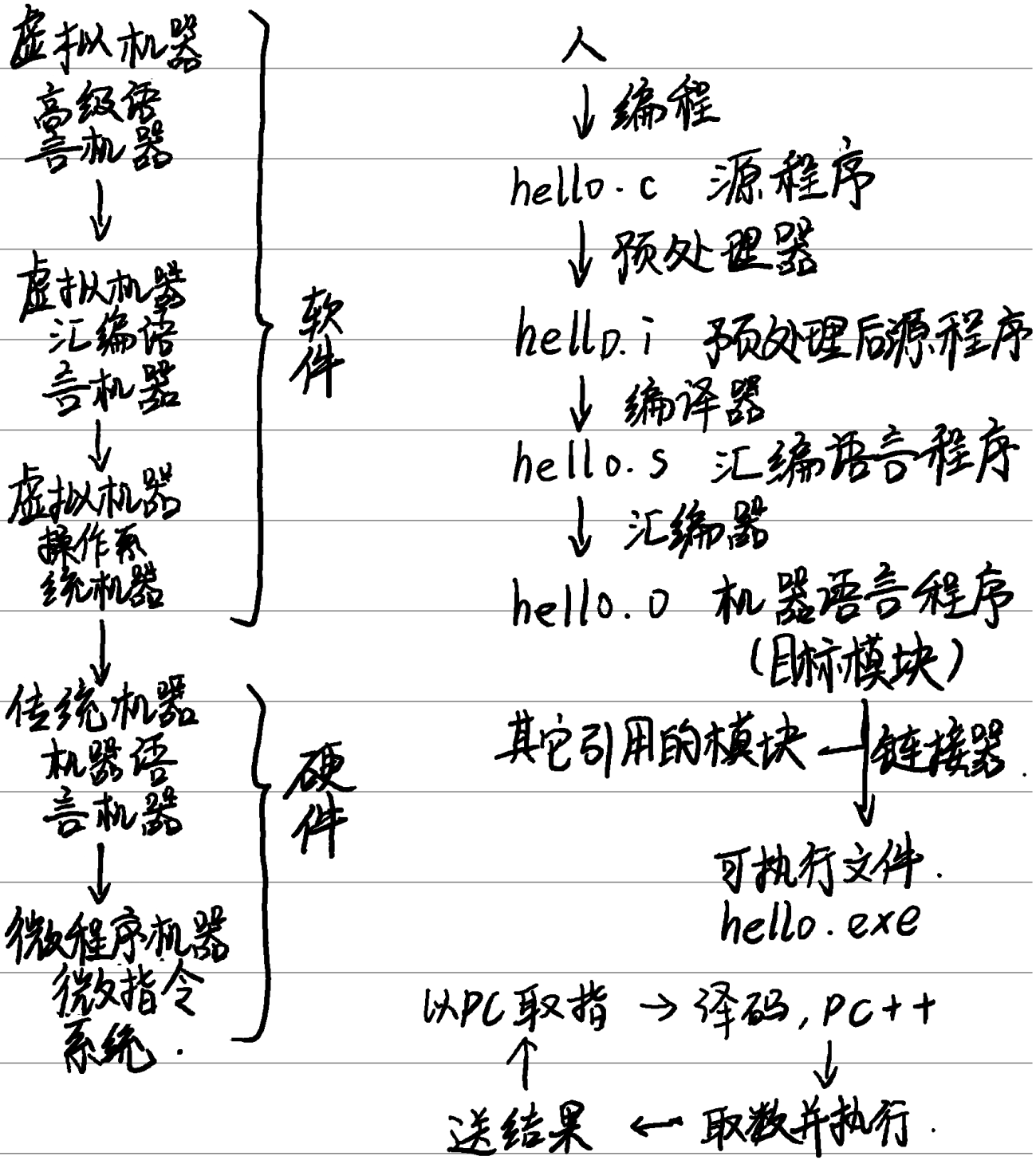
软件 { 应用软件
系统软件（包含 操作系统、数据库管理系统、标准程序库……）
↓基建

编译程序 { 高级语言 C/C++、Java、Python
↓汇编语言 助记符 （可能会跳过）
↓机器语言 二进制码
↑解释程序

指令集体系结构(ISA)：定义一台计算机可以支持哪些指令。

*同一个功能既可以用硬件实现，又可以用软件实现。(逻辑功能等价性)

系统的层次结构 & 工作原理


虚拟机器
高级语言机器
虚拟机器
汇编语言机器
虚拟机器
操作系统机器
传统机器
机器语言机器
微程序机器
微指令系统.
软件
硬件
人
编程
hello.c 源程序
预处理器
hello.i 预处理后源程序
编译器
hello.s 汇编语言程序
汇编器
hello.o 机器语言程序
(目标模块)
其它引用的模块
链接器.
可执行文件.
hello.exe
从PC取指 → 译码, PC++
送结果 ← 取数并执行.

性能指标

§ 主存储器.

总容量 = 存储单元数 × 存储字长

e.g. MAR 32位，MDR 8位.

总容量 $= 2^{32} * 8bit = 4\ GB$.

n个二进制位有 2^n 个状态.

$2^{10}: K \quad 2^{20}: M \quad 2^{30}: G \quad 2^{40}: T$

§ CPU.

主频：CPU内数字脉冲信号振荡频率.

↳clk频率 $\frac{1}{\text{CPU时钟周期}}$

CPI：执行每个指令需要周期数.

耗时：指令数 × CPI × 时钟周期（$\frac{1}{clk}$）

IPS：每秒执行指令数 $= \frac{\text{主频}}{CPI}$

FLOPS：每秒执行浮点运算数.

§ 整体性能指标（系统）

数据通路带宽：数据总线一次所能并行传送信息位数

吞吐量：系统在单位时间内处理请求数量

响应时间：请求到响应时间.

基准程序：用来测量计算机处理速度的一种应用程序.

§ 功耗

$P_{动} = \frac{1}{2} C \cdot V^2 \cdot f$.　（C为平均电容）

$C = \alpha C_L$

§ Amdahl 定律

改进后执行时间 = $\frac{\text{受改进影响的执行时间}}{\text{改进量}}$ + 不受影响执行时间

指令集 Instruction Set （MIPS）

add a, b, c // a = b + c

三地址格式

```
f = (g+h) - (i+j);
// add t0, g, h
// add t1, i, j
// sub f, t0, t1
```

MIPS 实例

add $S1, $S2, $S3 寄存器操作

sub 减法 addi 常数加法.

MIPS 32 bit 为一个字

大端对齐 (对人类更友好)

```c
g = h + A[8]
// lw $t0, 32($s3)   load word
// 4byte为1字, index 8 => 32
// index 0 => 0, index 1 => 4 ...
// add $s1, $s2, $t0
```

```c
A[12] = h + A[8]
// lw $t0, 32($s3)
// add $t0, $s2, $t0
// sw $t0, 48($s3)   store word.
```

- 编译器会尽量使用寄存器中数据.
- 处理内存中数据需将辅存中数据运至寄存器后再运算.

addi $s2, $s1, -1
// 立即数操作，减少不必要的加载
// 常量一般放内存里

MIPS有32个通用寄存器
- $0永远为0 => $zero 无法写入
- $0可用于清空寄存器
  add $t1, $zero, $zero
- $0可实现移动操作
  add $s2, $s1, $zero
- MIPS默认数据为带符号整数.
- lb, lh 扩展字节/半字
- beq, bne 扩展位移.

编号

8-15 → $t0 - $t7 临时寄存（调用）

16-23 → $s0 - $s7 保存寄存

24-25 → $t8 - $t9 临时寄存（被调用）

寄存器.

R0-R31 通用寄存器

PC 程序计数器

HI 乘法专用

LO 除法专用

R格式 (add $t0, $s0, $s1)

op	rs	rt	rd	sa	funt	
6	5	5	5	5	6	位数.
操作码	$s0地址	$s1地址	$t0地址	位移量	功能码add	

I格式

op rs rt immediate (常数/地址)

6 5 5 16 位

常数范围 $-2^{15} \sim 2^{15}-1$

J格式

op jump target

§ 逻辑运算.

<<	sll.
>>	srl
&	and, andi
|	or, ori
~	nor.

*shamt 用 0 补位 (移位)

and 按位取和

or 按位取或

*MIPS 无 not

a nor zero ⇒ NOT (a OR zero)

可达到取反效果.

§ 条件操作.

```
beq rs, rt, L1
// if (rs==rt) 执行 L1
bne rs, rt, L1
// if (rs!=rt) 执行 L1
j L1
// 无条件直接跳转.
```

```c
if (i==j) f=g+h;
else f=g-h;
/* 编译代码
    bne $S3, $S4, Else
    add $S0, $S1, $S2
    j exit
Else: sub $S0, $S1, $S2
Exit: ...
*/
```

```
C
while (save[i] == k) i += 1;
/* 编译代码
loop: sll $t1, $s3, 2    左移2位
      add $t1, $t1, $s6
      lw $t0, 0($t1)
      bne $t0, $s5, Exit
      addi $s3, $s3, 1
      j loop
Exit: ...
$s6 是 save 地址的地址
$s3 对应变量 i
$s5 对应变量 k
$t1 相当于指向 save[i] 的指针
*/
```

基本块

无嵌入分支、无分支目标的指令序列.

出口、入口惟一，一般无条件语句

slt rd, rs, rt

// if (rs < rt) rd = 1; else rd = 0;

slti rt, rs, constant

// if (rs < constant) rt = 1; else rt = 0;

slt $t0, $s1, $s2

bne $t0, $zero, L

// if ($s1 < $s2) branch to L

* blt, bge 不常用，硬件实现 <, ≥ 比实现 =, ≠ 慢；降低时钟.

Signed : slt 、slti

Unsigned : sltu , sltui

过程调用：结构化 、代码重用、易理解和维护.

步骤：① 参数放入寄存器

② 控制权转移到过程

③ 调取存储空间

④ 执行代码

⑤ 结果放入寄存器以供调用

⑥ 返回调用点

$a0 - $a3 4 - 7. 参数寄存器

$v0 - $v1 2 - 3 返回值寄存器

$gp 28 指向静态数据区

$sp 29 栈指针（指向当前栈顶）

$fp 30 帧指针（指向栈帧起始位）

$ra 31 返回地址

jal 标签：跳转到指定标签地址执行
把地址保存到$ra寄存器

jr（寄存器跳转）：jr $ra
将程序计数器设置为指定寄存器值

g → $a0, h → $a1, i → $a2,
j → $a3, f → $s0, 结果 → $v0

'''C

```
int leaf_exp (int g, h, i, j) {
    int f;
    f = (g+h) - (i+j);
    return f;
}
```



嵌套函数调用时，可能会覆盖参数寄存器，因此需存入栈后再安全覆盖.

111 C

```
int fact (int n) {
    if (n < 1) return 1;
    else return n * fact (n-1);
}
```

```
fact:
    addi $sp, $sp, -8      # adjust stack for 2 items
    sw   $ra, 4($sp)       # save return address
    sw   $a0, 0($sp)       # save argument
    slti $t0, $a0, 1       # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1     # if so, result is 1
    addi $sp, $sp, 8       #   pop 2 items from stack
    jr   $ra               #   and return
L1: addi $a0, $a0, -1      # else decrement n
    jal  fact              # recursive call
    lw   $a0, 0($sp)       # restore original n
    lw   $ra, 4($sp)       #   and return address
    addi $sp, $sp, 8       # pop 2 items from stack
    mul  $v0, $a0, $v0     # multiply to get result
    jr   $ra               # and return
```

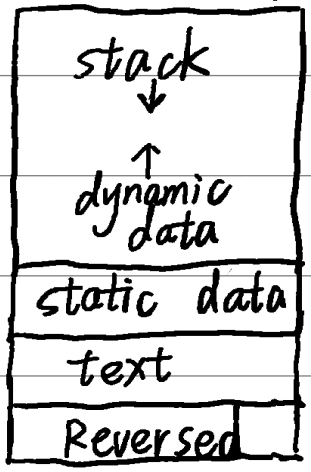
存放参数和返回地址

① 腾两个字的空间：存储 $ra、$a0

② 基本分支不用 lw.

* 所有递归程序都可以展开成循环指令程序.

存储器布局



自动存储（局部变量、返回地址参数）

动态分配内存
需手动释放内存

全局变量 & 静态变量

存放机器指令

字节、半字操作（文本、字符串处理）

lb rt, offset(rs) #Load byte

lh rt, offset (rs) # load halfword

符号扩展.

lbu, lhu 无符号扩展

sb store byte sh store halfword.

l- 从内存读取到寄存器

s- 将寄存器内容写入内存.

```C string Copy Example.

```
//字符串复制函数.
void strcpy (char x[], char y[]){
    int i;
    i=0;
    while ((x[i]=y[i])!='\0'){
        i++;
    }
}
// x,y地址为 $a0, $a1
// i 储存在 $s0.
```
```

```
strcpy:
    addi    $sp, $sp, -4
    sw      $s0, 0($sp)     #存入
    add     $s0, $zero, $zero   #i=0
L1: add     $t1, $s0, $a1
    # 把y[i]的地址存入 $t1
    lbu     $t2, 0($t1)
    # 把y[i]的数据读入 $t2
    add     $t3, $s0, $a0
    # 把x[i]的地址存入 $t3
    sb      $t2, 0($t3)
    # 把 y[i] ($t2) 写入 x[i] ($t3)
    beq     $t2, $zero, L2  #跳出循环
    j       L1
L2: lw      $s0, 0($sp)
```

```
addi   $sp, $sp, 4    #释放空间
jr     $ra            #return
```

MIPS 指令长度固定为 32 位.
指令中立即数只有 16 位

§ 将 32 位常数加载到寄存器

① lui rt, constant
#将 constant 高 16 位存入 rt 高 16 位.
#低 16 位清 0.

② ori rt, rs, constant
#低 16 位按位取或.
#保持高 16 位不变.

分支指令相对寻址.

目标地址 = PC + 偏移量 ×4

↑ 基准

跳转指令的直接选址.

目标地址 = PC [31:28] : (addres ×4)

寻址太远时.

beq $s0, $s1, L1

⇒ 汇编器自动条件取反.

bne $s0, $s1, L2

j L1

L2:

§ 原子操作

用于避免数据竞争.

原子交换

```
atomic_swap:
    ll   $t1, 0($a0)
    sc   $t0, 0($a0)
    beqz $t0, atomic_swap
    jr   $ra
```

// ll rt, offset(rs) 加锁机制

从 rs + offset 读取到 rt.

sc rt, offset(rs)

将 rt 的值存入 rs + offset

适用于多核的处理场景，不被其它进程打断

原子加

```
atomic_add_one:
    ll    $t0, 0($s1)
    # 加载链接，将内存读到$s1
    addi  $t0, $t0, 1
    sc    $t0, 0$(s1)
    # 条件存储，成功时$t0为1，
    # 失败则为0.
    beq   $t0, $zero. atomic_add_one
    # 成功时，原子操作完成，
    # 可执行后续指令. 否则重试.
```

§ 汇编器处理

真实指令：CPU硬件直接支持

伪指令：汇编器虚构，实际不存在

move → add.

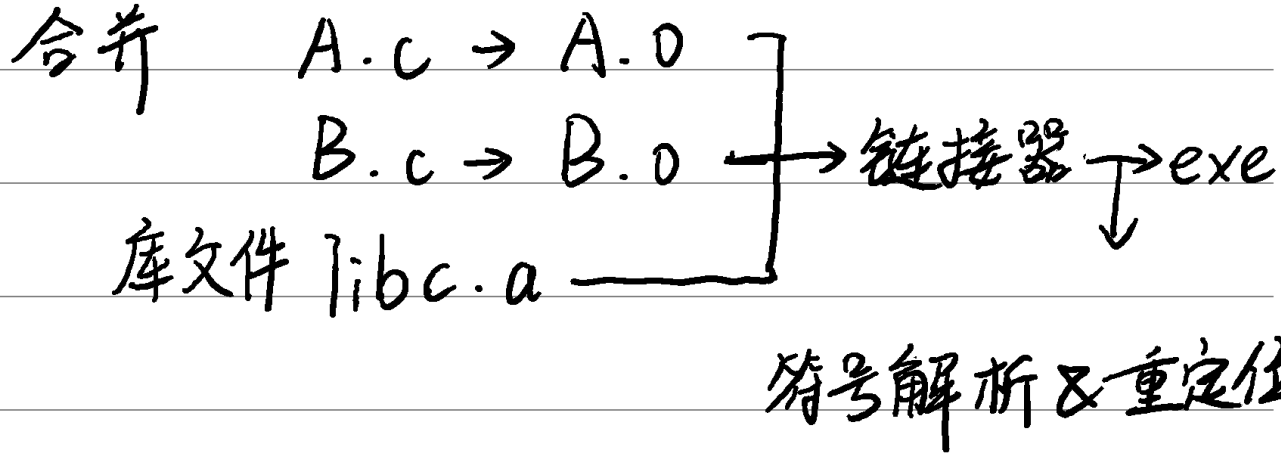
li → addi / lui + ori

blt → slt + bne

$at 专门保留给汇编器使用的汇编器临时寄存器

§ 目标文件

源文件(.c) → 编译器/汇编器 → 目标文件(.o) → 链接器 → 可执行文件 → 最终程序



加载器工作流程

① 读取文件头，确定段大小

② 创建虚拟地址空间

text、data、堆、栈

③ 复制 text 和初始化 data 到内存

④ 设置栈上参数

⑤ 初始化寄存器

⑥ 跳转到启动例程

#静/动态链接

静态：链接时直接复制到可执行文件中，不依赖外部库.

缺点：库更新时，须重新编译链接

代码量膨胀

动态：可执行文件中只记录库的引用

代码须可重定位

自动获取新版本

共享内存.

特性	静态链接	动态链接
可执行文件大小	大（包含库代码）	小（只有引用）
内存占用	每个程序独立拷贝	多个程序共享一份库
库更新	需重新链接	自动使用新版本
启动速度	快（无需解析）	稍慢（运行时链接）
依赖管理	无依赖	需要库文件存在
典型后缀	`.a` (UNIX), `.lib` (Windows)	`.so` (UNIX), `.dll` (Windows)

动态链接：库例程只在第一次被调用时才真正链接
（lazy linkage）

.Java
↓
javac（编译器）
↓
Class files ← 字节码文件（.class）
(Java bytecodes)
↓
即时编译器　　字节码不依赖特定CPU架构
↓
本地机器码

冒泡排序.

```c
void swap (int v[], int k) {
    int tmp;
    tmp = v[k];
    v[k] = v[k+1];
    v[k+1] = tmp;
}
```

```
/*  v -> $a0   k -> $a1
    tmp -> $t0
```

```
swap : sll  $t1, $a1, 2  # $t1 = 4*k
       add  $t1, $t1, $a0  #寻址
```

```
lw   $t0,  0($t1)
lw   $t2,  4($t1)    # $t2 = v[k+1]
sw   $t2,  0($t1)
sw   $t0,  0($t2)
jr   $ra
```

叶子函数不用栈（$sp） */

```
void sort (int v[], int n) {
    int i, j;
    for (i=0; i<n; i+=1) {
        for (j=i-1; j>=0 && v[j] >
                v[j+1]; j-=1) {
            swap(v, j);
        }
```

}

}

/* v → $a0 n → $a1 i → $s0
j → $s1

```
        move $s2, $a0           # save $a0 into $s2
        move $s3, $a1           # save $a1 into $s3
        move $s0, $zero         # i = 0
for1tst: slt  $t0, $s0, $s3     # $t0 = 0 if $s0 ≥ $s3 (i ≥ n)
        beq  $t0, $zero, exit1  # go to exit1 if $s0 ≥ $s3 (i ≥ n)
        addi $s1, $s0, -1       # j = i - 1
for2tst: slti $t0, $s1, 0       # $t0 = 1 if $s1 < 0 (j < 0)
        bne  $t0, $zero, exit2  # go to exit2 if $s1 < 0 (j < 0)
        sll  $t1, $s1, 2        # $t1 = j * 4
        add  $t2, $s2, $t1      # $t2 = v + (j * 4)
        lw   $t3, 0($t2)        # $t3 = v[j]
        lw   $t4, 4($t2)        # $t4 = v[j + 1]
        slt  $t0, $t4, $t3      # $t0 = 0 if $t4 ≥ $t3
        beq  $t0, $zero, exit2  # go to exit2 if $t4 ≥ $t3
        move $a0, $s2           # 1st param of swap is v (old $a0)
        move $a1, $s1           # 2nd param of swap is j
        jal  swap               # call swap procedure
        addi $s1, $s1, -1       # j -= 1
        j    for2tst            # jump to test of inner loop
exit2:  addi $s0, $s0, 1        # i += 1
        j    for1tst            # jump to test of outer loop
```

```
sort:    addi $sp,$sp, -20      # make room on stack for 5 registers
         sw $ra, 16($sp)        # save $ra on stack
         sw $s3,12($sp)         # save $s3 on stack
         sw $s2, 8($sp)         # save $s2 on stack
         sw $s1, 4($sp)         # save $s1 on stack
         sw $s0, 0($sp)         # save $s0 on stack
         …                      # procedure body
         …
         exit1: lw $s0, 0($sp)  # restore $s0 from stack
         lw $s1, 4($sp)         # restore $s1 from stack
         lw $s2, 8($sp)         # restore $s2 from stack
         lw $s3,12($sp)         # restore $s3 from stack
         lw $ra,16($sp)         # restore $ra from stack
         addi $sp,$sp, 20       # restore stack pointer
         jr $ra                 # return to calling routine
```

指令数和CPI不能单独衡量程序运行时间

编译器优化依赖算法

§ 指针比数组索引更高效.

数组索引需计算偏移;

指针访问直接使用内存地址

```
clear1(int array[], int size) {
  int i;
  for (i = 0; i < size; i += 1)
    array[i] = 0;
}
```

```
clear2(int *array, int size) {
  int *p;
  for (p = &array[0]; p < &array[size];
       p = p + 1)
    *p = 0;
}
```

```
       move $t0,$zero   # i = 0
loop1: sll $t1,$t0,2    # $t1 = i * 4
       add $t2,$a0,$t1  # $t2 =
                        #   &array[i]
       sw $zero, 0($t2) # array[i] = 0
       addi $t0,$t0,1   # i = i + 1
       slt $t3,$t0,$a1  # $t3 =
                        #   (i < size)
       bne $t3,$zero,loop1 # if (…)
                           # goto loop1
```

```
       move $t0,$a0     # p = & array[0]
       sll $t1,$a1,2    # $t1 = size * 4
       add $t2,$a0,$t1 # $t2 =
                        #   &array[size]
loop2: sw $zero,0($t0) # Memory[p] = 0
       addi $t0,$t0,4   # p = p + 4
       slt $t3,$t0,$t2 # $t3 =
                        #(p<&array[size])
       bne $t3,$zero,loop2 # if (…)
                           # goto loop2
```

強度削弱

x = i * 4; ⇒ x = i << 2;

编译器会自动将数组索引优化为指针版本.

```
for (int i=0; i<100; i++) {
    sum += arr[i];
}
```

```
int *p = arr;
int *end = arr + 100;
while (p < end) {
    sum += *p++;
}
```

MIPS vs ARM

特性	ARM	MIPS
推出年份	1985	1985
指令大小	32 位固定	32 位固定
地址空间	32 位平坦寻址	32 位平坦寻址
数据对齐	要求对齐	要求对齐
数据寻址模式	9 种	3 种
通用寄存器	15 个 × 32 位	31 个 × 32 位
输入/输出	内存映射 I/O	内存映射 I/O

ARM条件码

Negative 最高位为 1

Zero 所有位为 0

Carry 加法进位 & 减法借位

oVerflow 符号溢出

Name	Definition	ARM v.4	MIPS
Load immediate	Rd = Imm	mov	addi, $0,
Not	Rd = ~(Rs1)	mvn	nor, $0,
Move	Rd = Rs1	mov	or, $0,
Rotate right	Rd = Rs i >> i $Rd_{0...i-1} = Rs_{31-i...31}$	ror	
And not	Rd = Rs1 & ~(Rs2)	bic	
Reverse subtract	Rd = Rs2 - Rs1	rsb, rsc	
Support for multiword integer add	CarryOut, Rd = Rd + Rs1 + OldCarryOut	adcs	—
Support for multiword integer sub	CarryOut, Rd = Rd – Rs1 + OldCarryOut	sbcs	—

x86 指令采用 **双操作数** 格式：`操作 目标，源`

源/目标操作数	第二源操作数	示例
寄存器	寄存器	`add eax, ebx`
寄存器	立即数	`add eax, 42`
寄存器	内存	`add eax, [ebx]`
内存	寄存器	`add [eax], ebx`
内存	立即数	`add [eax], 42`

算术指令可直接操作内存.
格式位数长度可变.

地址 = 基址 + 偏移.
= 基址 + 缩放 × 索引
(+偏移)

scale	2^scale	用途
0	1	字节数组
1	2	短整型数组（2 字节）
2	4	整型数组（4 字节）
3	8	长整型数组（8 字节）

x86是 CISC 架构 (复杂指令集)
指令不同，格式不同，长度不同.

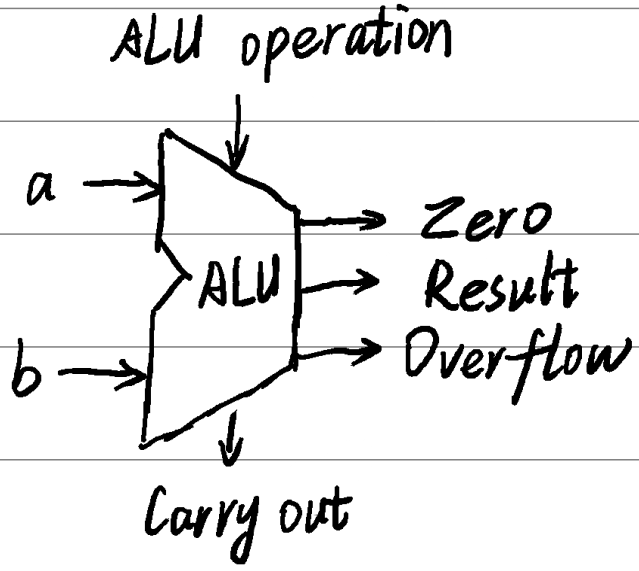
计算机算术

基本算术/逻辑指令

add sub

and or

用 sub 作差比较即可实现"<"



完整 ALU

32位加法器

input A, B.

control lines Binvert Operation

Carryin

output Result Overflow

- 简单加法器：行波进位加法器
  串行运行，不适用于高速应用场景

两级逻辑 = 第一级与门 + 第二级或门
门电路输入数量徒增

方案	门数量	门输入数	延迟
两级逻辑	最少	极大（不可实现）	最小（2级）
行波进位	少	小	大（O(n)）
超前进位	中	中	中（O(log n)）

① 超前进位加法器

在求和之前并行计算所有进位，无需等待低位进位逐级传递

缺点：电路复杂，门输入数量大

② 进位跳跃加法器.

将加法器分成若干块，若某个

块内P全为1 即 ($A_i + B_i$全为1)，进位可以直接跳过该块.

缺点：加速取决于跳过块数量

③进位选择加法器

同时计算进位为0/1的两种结果，等真实进位到来时直接选择正确结果.

缺点：硬件成本高.

§ 加法器逻辑（进位）

输出 $C_{i+1} = (a_i * b_i) + (a_i + b_i) * c_i$

生成 $g_i = a_i * b_i$

传播 $p_i = a_i + b_i$

传统标量运算：一次只能做64位加法

SIMD：可做8个8位/4个16位/…加法

└ 原理：内部插进位阻断.

#饱和运算

11111111 + 00000001 = 00000000

绕回

饱和运算 255+1 = 255 停在最大值

unsigned 0-1 = 0 停在最小值

进位计数制

{ 十、二、八、十六进制
  进制间相互转化
  真值和机器数.

真值：符合人类习惯的数字

机器数：数存储机器内的形式.

原、反、补、移码

定点数的编码表示

定点数：小数点位置固定

浮点数：小数点位置不固定

- 定：996.007 常规计数
- 浮：9.96007×10^2 科学计数.

- 无符号 → 相当于数的绝对值
- 有符号：原、反、补、移码.

§ 无符号数（不讨论小数）.

k位二进制范围：$0 \sim 2^k-1$

unsigned int / long / longlong

§ 有符号数.

定点整数.

符号位 + 数值部分 + 小数点(隐)

定点小数

符号位 + 小数点位置(隐) + 数值部分(尾数)

机器字长为 k ⇒ 尾数占 $k-1$ 位.

原码 { 整数范围 $-(2^{k-1}-1) \sim (2^{k-1}-1)$
小数范围 $-(1-2^{-k-1}) \sim (1-2^{-k-1})$

含 +0 和 -0.

反码表示范围和原码相同

补码 { 整数范围 $-2^{k-1} \sim 2^{k-1}-1$
小数范围 $-1 \sim 1-2^{-n}$.

移码（只表示整数）

补码符号位取反

范围 $-2^{k-1} \sim 2^{k-1}-1$

* 便于比较真值大小.

$[0]_{移} = 10000000$

e.g. $[x]_{原} = 10001101$ -13

$[x]_{反} = 10001101$

$\Downarrow$

01110010 -114

$[x]_{补} = 10001101$

$\Downarrow$

10001100

$\Downarrow$

01110011 -115

$[x]_{移} = 10001100$ 13.

正数 + 负数 不能直接用原码

正数 ↓ 正数

计算机用加法代替减法.

- 在模 12 的群中，-3，+9 等效，
- 绝对值相加为模的两个数互为补数

14 - 14. (8bit) ⇒ 模 2^8

⇒ $14 + (2^8 - 14)$

↳ 11110010 (-14的补码)

补码作用：可将减法操作转化为等价加法操作，硬件成本↓

移码作用：方便比较大小.

C 语言数制类型转换

C语言中 int、short、long 定点整数以补码形式存储（unsigned 无符号）

```
short x = -4321.
unsigned short y = (unsigned short)x
//  x : 1110 1111 0001 1111
//  y : 1110 1111 0001 1111
//  y = 61215
```

```
int a = 165337 , b = -34991
short c = (short)a , d = (short)b;
// 长整数变短整数，高位截断
```

short x = -4321

int m = x.

//短整数变长整数，符号扩展.

§零扩展 & 符号扩展.(短 => 长)

unsigned short 90 -> unsiged int

01011010 => 00000000 01011010

无符号数直接 零扩展

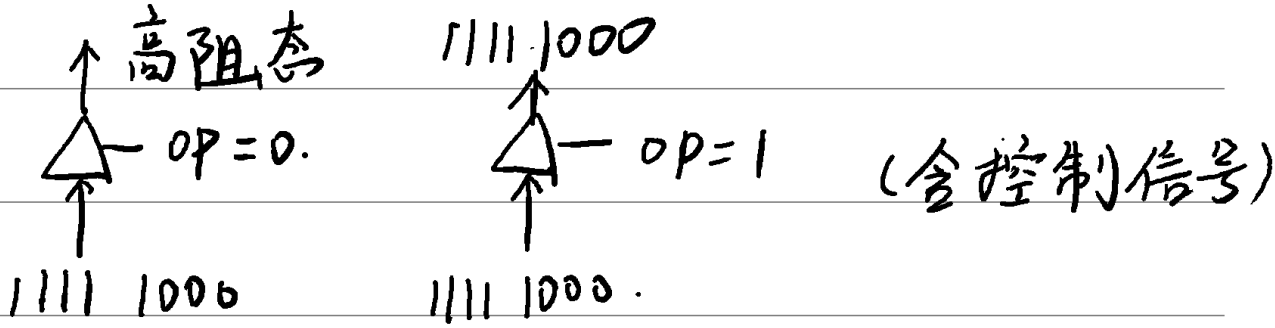
short -90 => int.

10100110 (符号位为1，补1)

=> 1,11111111 01001110

有符号数使用符号扩展.

三、态门

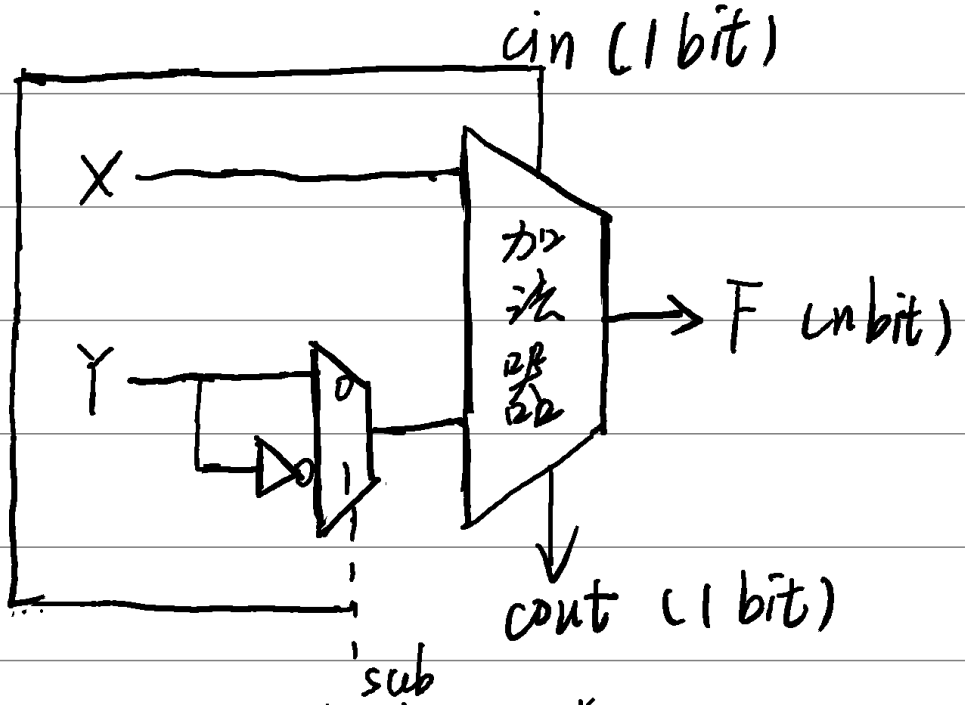


(断路 0V).



非门不含控制信号.

补码加减运算



控制减法运算.

sub为1, cin为1

OF: 溢出 (带符号数).

SF: 符号 (带符号数结果正负)

ZF: 零标志, 运算结果是否为0.

CF: 进/借位 (无符号数运算溢出)

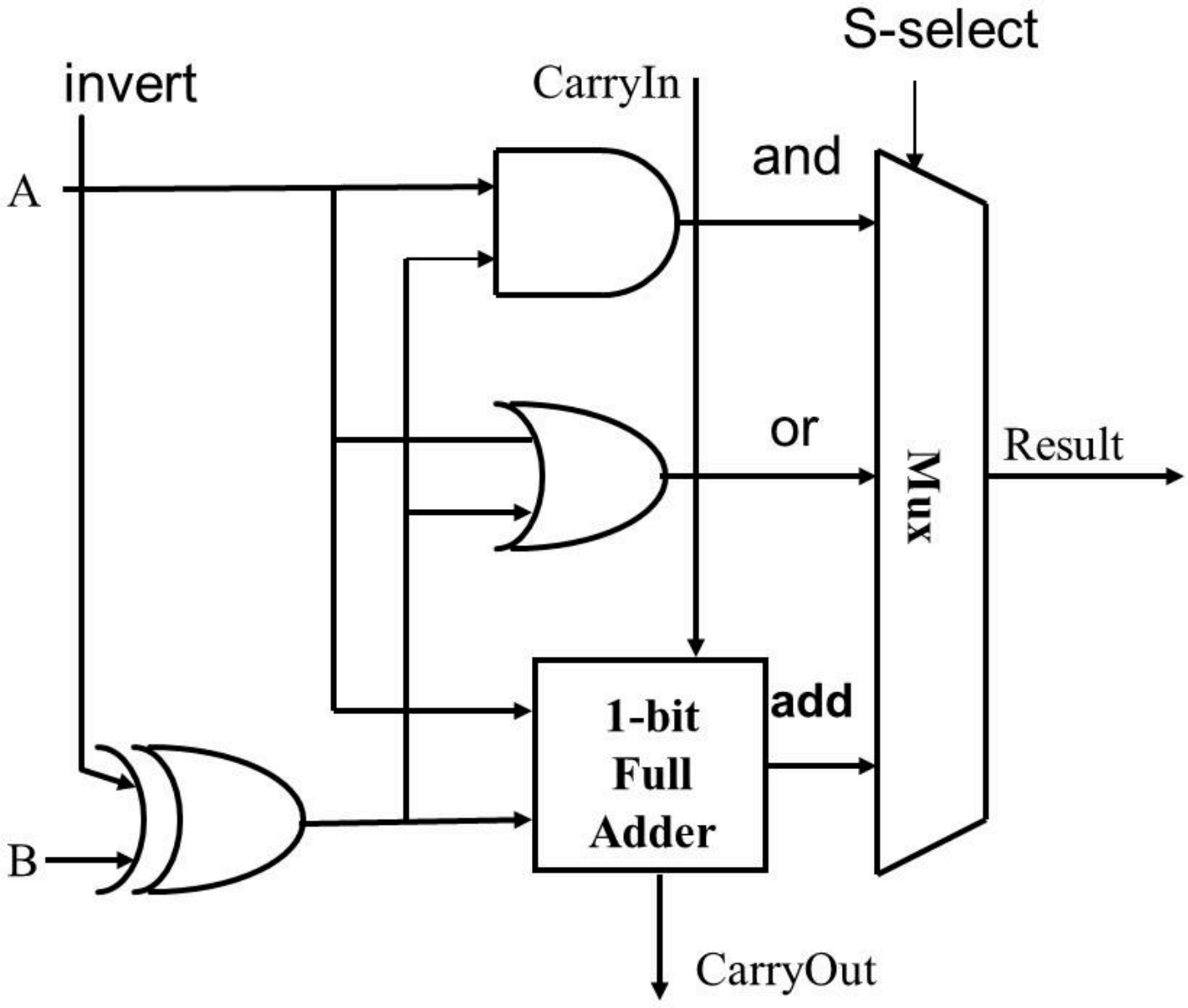
$OF = C_n \oplus C_{n-1}$

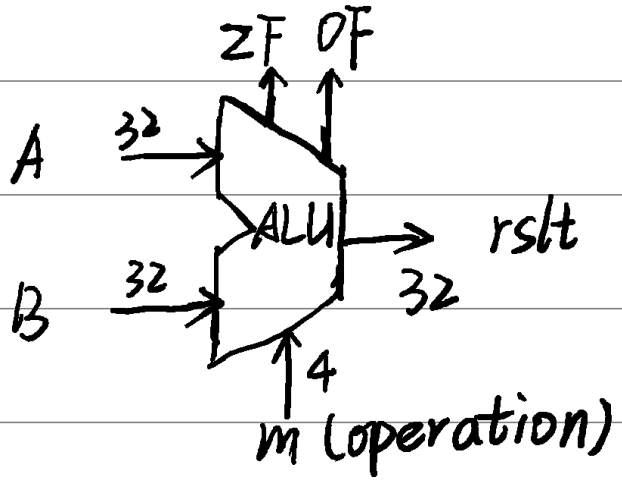
$SF = S_n$.

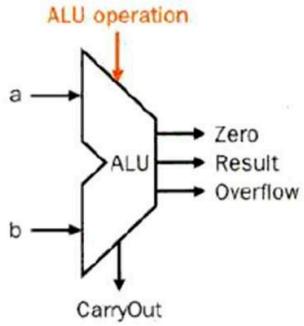
$ZF = S_n + \cdots\cdots + S_2 + S_1$

$CF = cout \oplus cin = C_n \oplus C_0 = C_n \oplus Sub$

{ 加 cin为0，cout为1溢出，无符号.
减 cin为1，cout为0溢出，







000	And	按位与
001	Or	按位或
010	Add	加
110	Sub	减
111	Slt	比大小

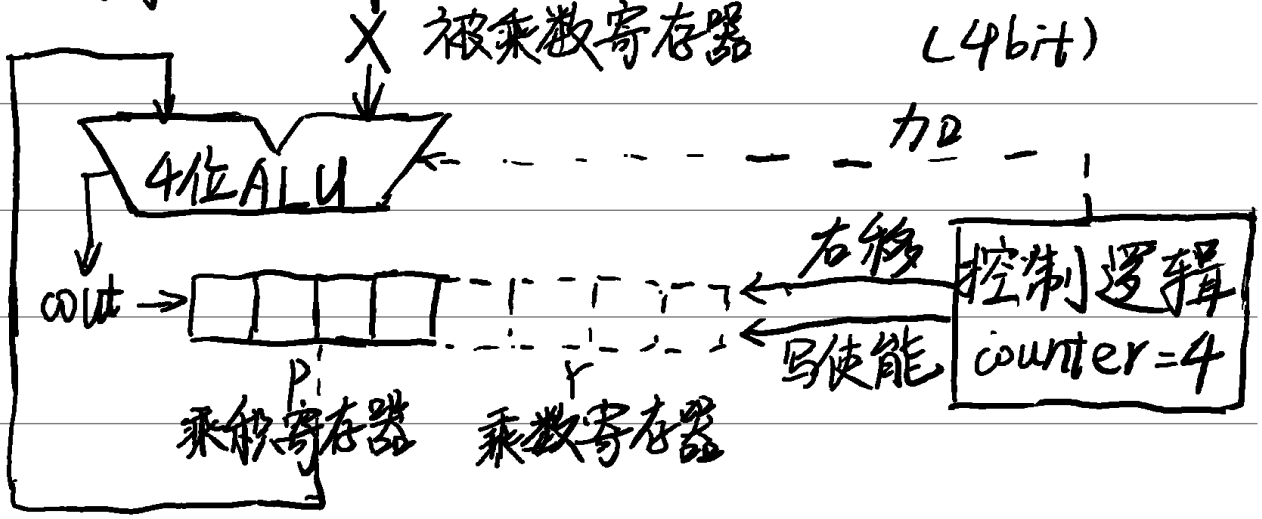
乘法运算

手算

```
  1101
 x1011
 ----
  1101
 1101
1101
-------
10001111
```

```
   1101
  x1011
  -----
   0000   部分积
 + 1101
  -----
   1101.
+ 1101
------
 100111
+0000
------
 100111
+1101
------
10001111   最终积
```

§ 无符号数乘法



$Y_1 = 1$，执行加法后，右移.

$Y_1 = 0$，直接右移.

counter -= 1

* 乘法运算用2n位暂存，通常仅保留低n位作为乘积结果，可能会溢出

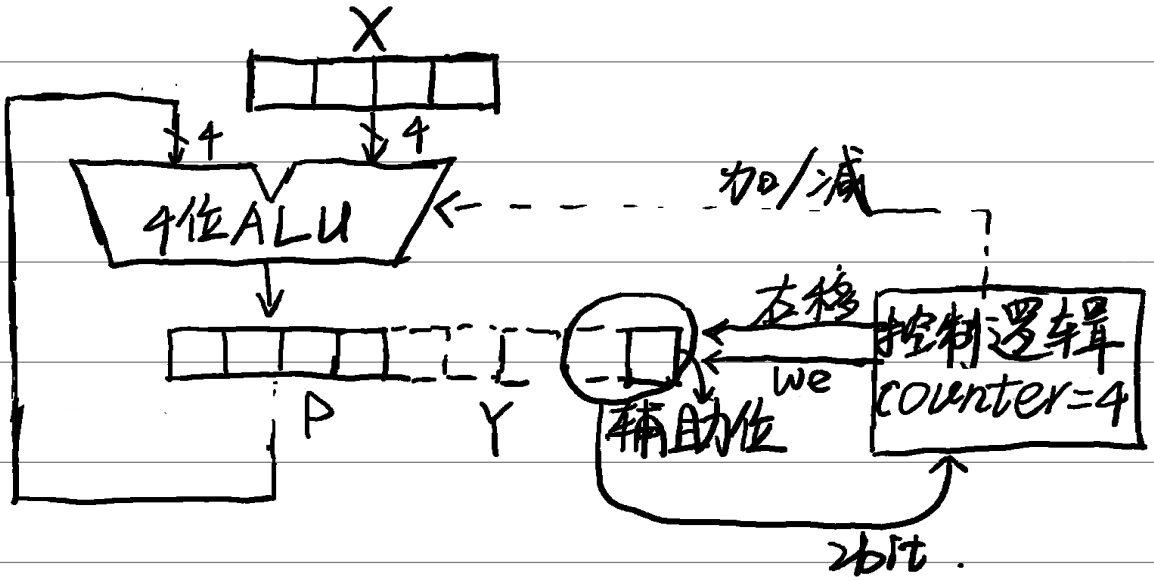
溢出判断：高n位不全为0则发生溢出

OF置为1.

应对：可执行“溢出自陷指令”.

§ 带符号数乘法.

C语言 int型，32 bit 带符号整数，用补码表示.



① 辅助位初始化为 0

② 00 ⇒ +0.　　(Y, +辅助位)

01 ⇒ +[X]补

10 ⇒ −[X]补

11 ⇒ +0.

③ P、Y、辅助位右移，Cn −= 1

④ P的补位直接补原最高位.

减法 $-1101 \Rightarrow +0010+1$

处理. $\Rightarrow +0011$.

*带符号整数乘法，通常仅保留 n 位作为结果，发生溢出，OF置为1.

判断： $n \sim 2n$ 位完全相同.

处理：执行溢出自陷处理.

Booth乘法：补码一位乘法

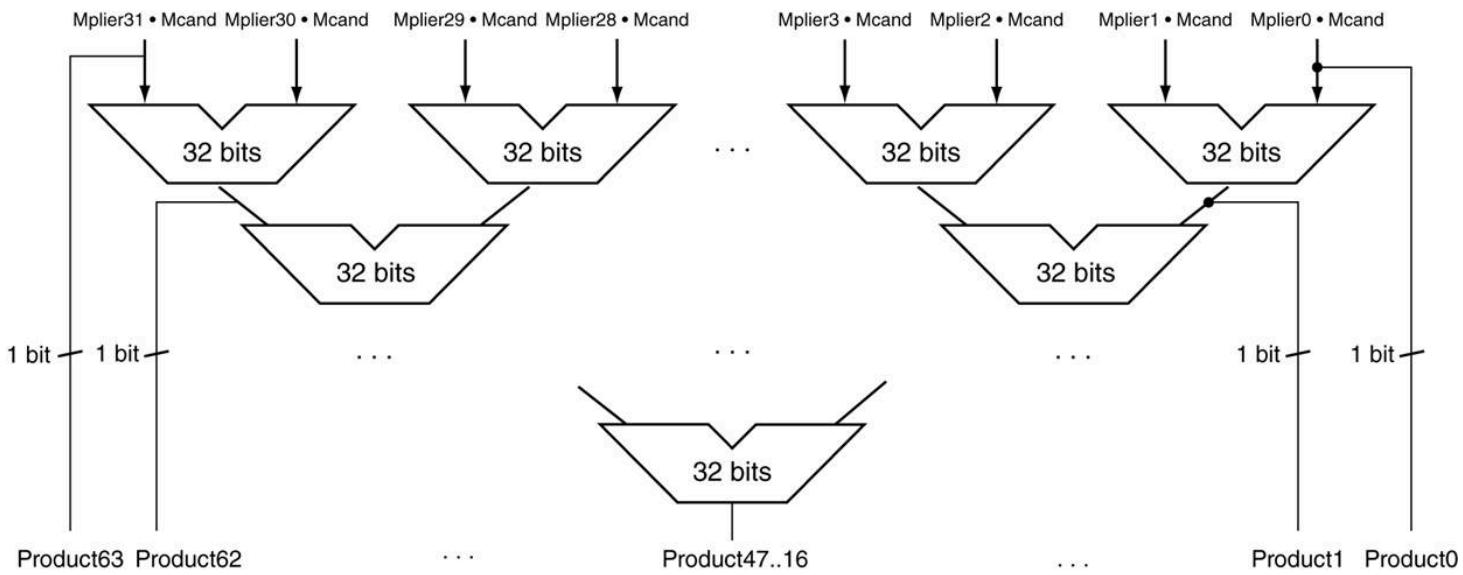
§ 其它乘法运算方式（无符号数）

1. 改为两位乘法：每次处理 2bit
   只需要 $\frac{n}{2}$ 个时钟即可完成运算.
2. 阵列乘法器（1个时钟）
   快速乘法器

3. 只用加/减/逻辑运算(极慢)

```
unsigned int fun (unsigned int x,
                  unsigned int y){
    unsigned int rst = 0;
    for (int i=0; i<32; i++){
        unsigned int bit = y & 1;
        if (bit) rst += x;
        x <<= 1, y >>= 1;
    }
    return rst;
}
```

并行加法器树



mult rs, rt　　有符号

multu rs, rt　　无符号

64位结果存入{HI, LO}

mfhi $t0　　读高32位

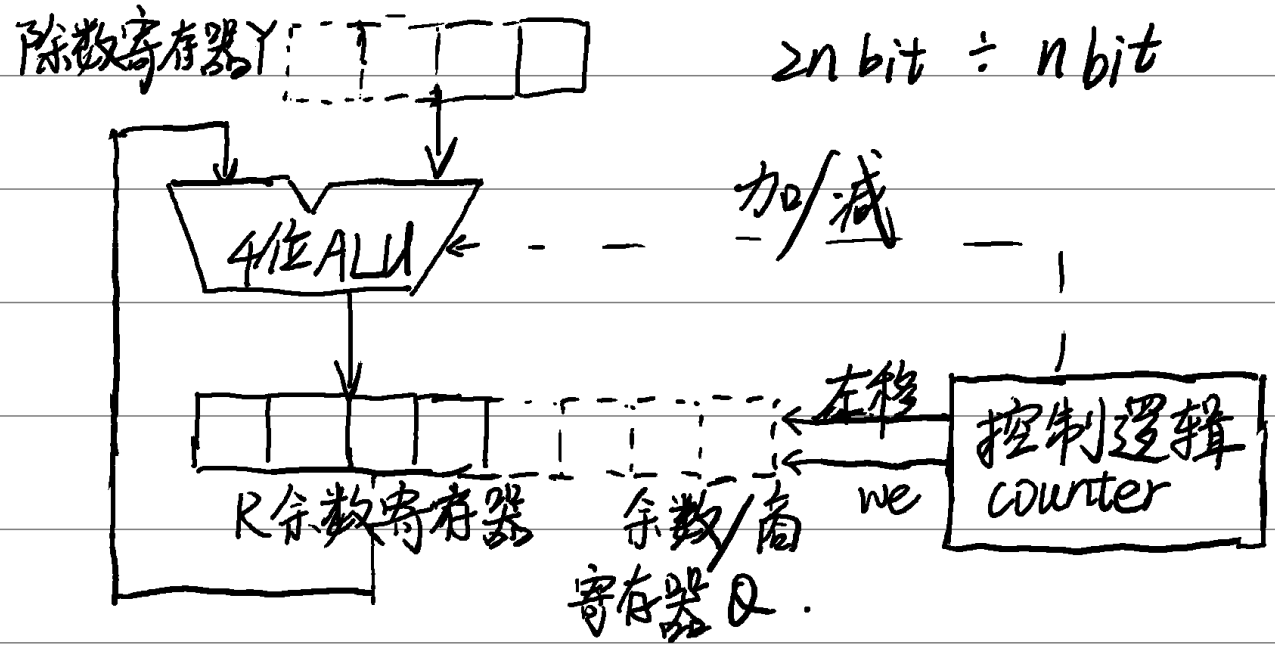
mflo $t1　　读低32位

mul $t0, $s0, $s1

只取低32位

溢出判定
- unsigned　HI = 0
- signed
  - HI = 0.
  - HI = 0xffffffff

无符号除法



[恢复余数法]

① 除数为 0，throw异常.

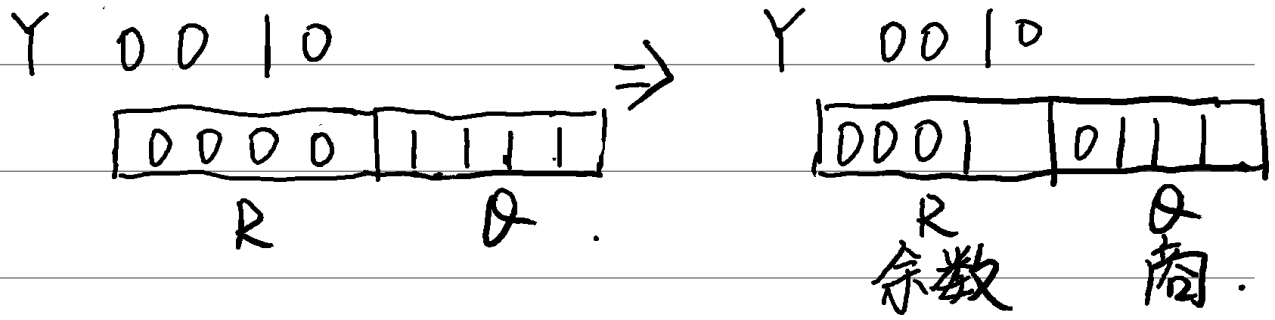
② 被除数 < 除数，商 0.

③ 上商：减法比较. R ≥ Y，上商 1.

第1位商不为0，"商溢出"，停止.

④ 先左移，空位上商（先减比较，再加还原），counter--.

性质：左移后R自动变为中间余数。
商溢出指商的位数大于n。
商1时不用还用。



e.g. unsigned int x = 985211,
y = 666, q = x / y, r = x % y;

寄存器：R全零，Q为x二进制(零补全)

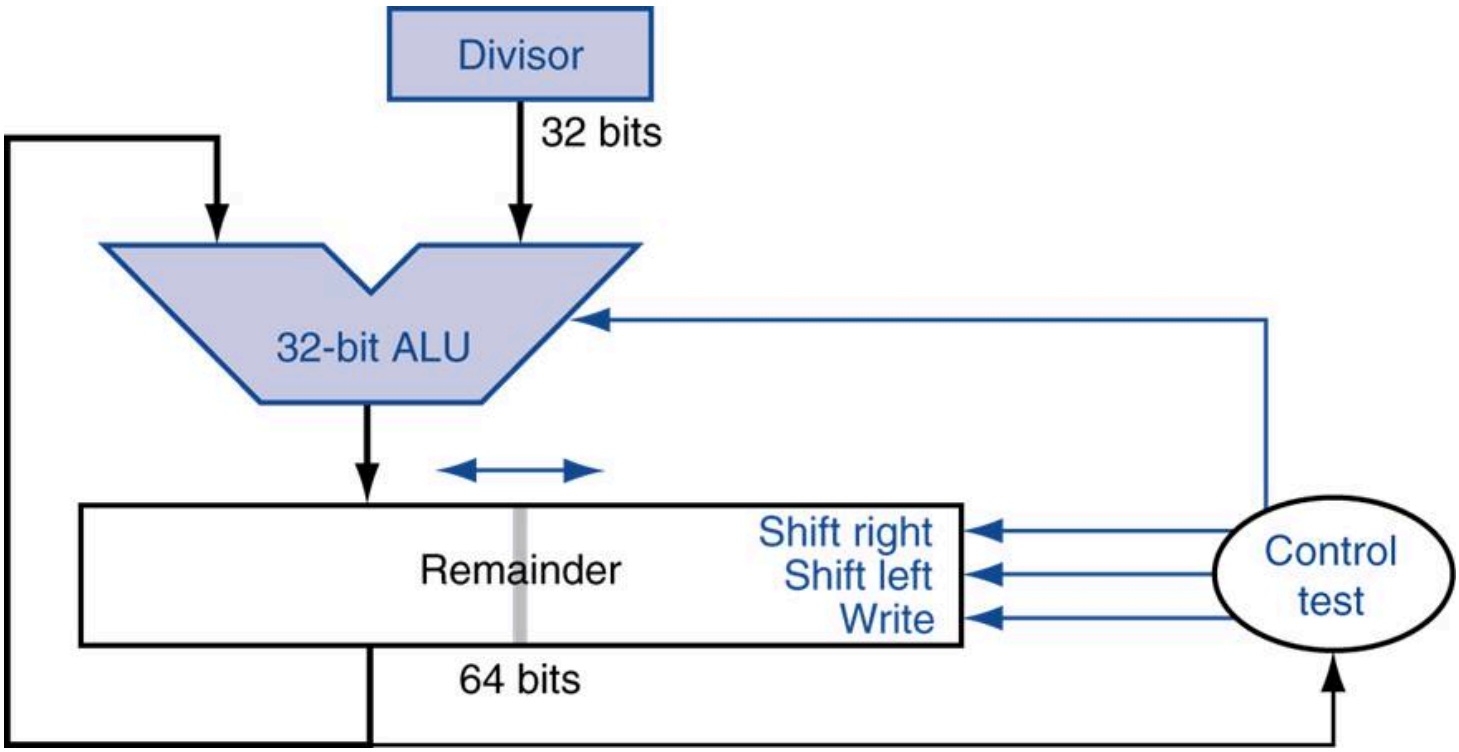
⇓

R为余数，Q为商.

双精度除法：2n bit ÷ n bit

单精度除法：n bit ÷ n bit .（不会溢出）

*除数为0，商溢出均为除法错异常（x86中）



HI：32-bit 余数.

LO：32-bit 商

div $s0, $s1	有符号
divu $s0, $s1	无符号.
mfhi $t0	读余数.

mflo $t1　　读商

异常情况

除数为0 } MIPS硬件行为
溢出 } 均未定义

须软件进行检查.

除法器加速（无并行）

每一步减法取决上一步余数

商位的确定是串行依赖.

SRT除法

① 根据被除数和余数的高位几位，查一个预先设计好的表格，直接猜出下一位/多位商

② 猜测商×除数，从余数中减去

③ 猜大或猜小，后续修正.

优势：总体步数减少.

现代处理器

SRT基数4或基数16除法器

§ 加减交替法（不恢复余数）

$0.1001 \div 0.1101$

1.0010 先左移，个位商零.

-0.1101

0.0101 >0，商1

0.1010

-0.1101

-0.0011 <0，商0

-0.0110

$+0.1101$

0.0111 >0，商1.

$$
\begin{array}{r}
0.1110 \\
-0.1101 \\
\hline
0.0001
\end{array}
\quad > 0，商 1
$$

商为 0.1011 ，余数为 0.00000001

IEEE 754 浮点数.

C 语言 float、double 符合 IEEE 754 标准表示

$+3.026 \times 10^{11}$（阶码）

符号 尾数 基数

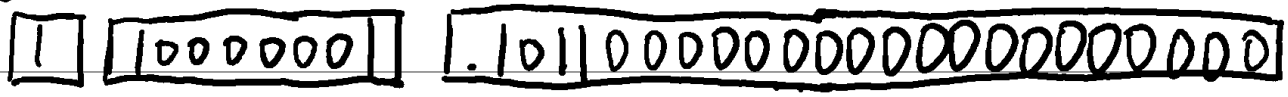
二进制个位一定为 1

二进制 $-110.11 \Rightarrow -1.1011 \times 2^2$.

float	符号	阶码	尾数
	1bit.	8bit	23bit

double	符号	阶码	尾数
	1bit	11bit	52bit.

e.g. -1.1011×2^2.



① 尾数小数点前的 1 省略（隐含）.

② 阶码用移码形式表示，偏置为 127

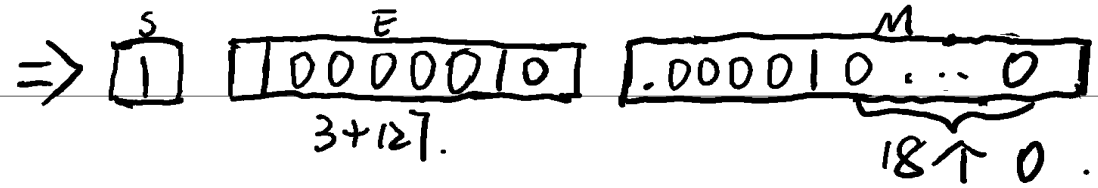
double 型偏置为 $2^{11-1}-1=1023$.

e.g.1. -8.25 Dec 转换为 IEEE 754.

float / double.

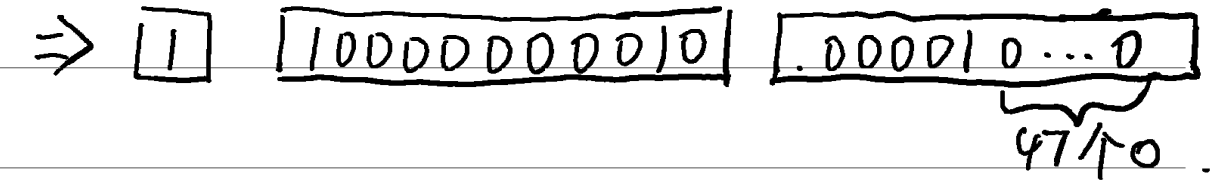
(1) $-8.25 \Rightarrow -1000.01$

$\Rightarrow -1.00001 \times 2^3$.



$\Rightarrow$ C104 0000 H (十六进制)

(2)



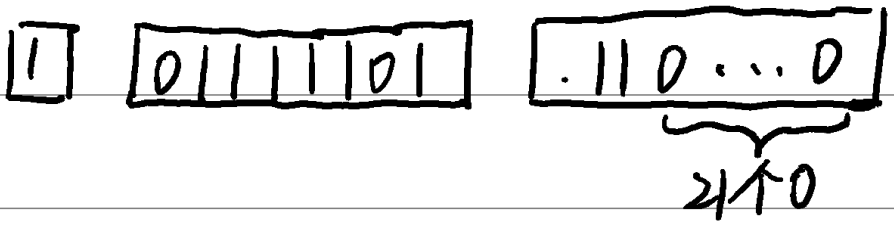
$\Rightarrow$ C020 8000 0000 0000 H.

float: $X=(-1)^S \times (1.M) \times 2^{E-127}$

e.g.2 $-0.4375 \Rightarrow$ IEEE 754 float

$-0.4375 \Rightarrow -0.0111$

$\Rightarrow -1.11 \times 2^{-2}$

$\Rightarrow$ 

$\Rightarrow$ BEE0 0000 H.

小数 $\Rightarrow$ 十进制

$0.4375 \times 2 = 0.875$. 0

$0.875 \times 2 = 1.75$ 1

$0.75 \times 2 = 1.5$ 1

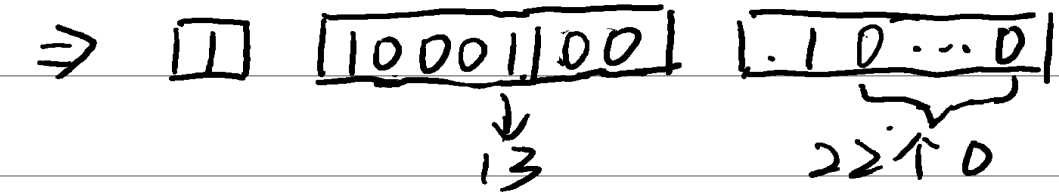
$0.5 \times 2 = 1.0$ 1

$\Rightarrow 0.0111$

e.g.3. IEEE754 C640 0000H ⇒ 小数.

C640 0000H. ⇒ 1100 0110 0100

后接20个0.



⇒ -1.5×2^{13}

§ 表示范围 & 几种特殊状态.

规定：仅当阶码不全为0且不全为1时，表示一个"规格化浮点数".

① 阶码、尾数全零 ⇒ 0

有正零和负零.

② 阶码全零，f不为0 ⇒

$\pm 2^{-126}(0.f)$ / $\pm 2^{1022}(0.f)$.

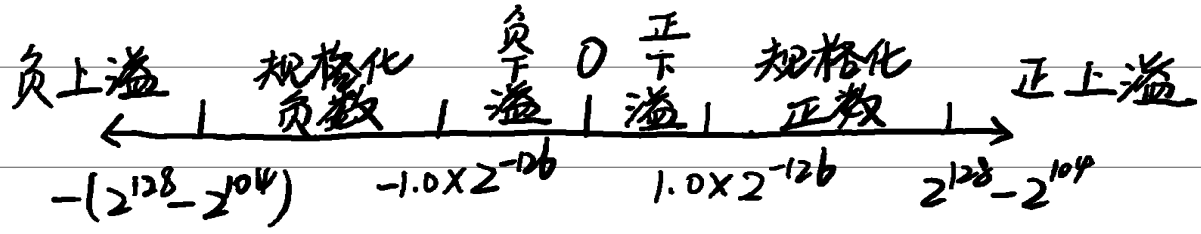
③ 阶码全1，尾数全0 ⇒ ±∞.

④ 阶码全1，f≠0 ⇒ NaN.

float 阶码真值范围 -126 ~ 127

最大值 $2^{128}-2^{104}$

最小值 $-(2^{128}-2^{104})$



上溢出处理：将运算结果设为 +∞ 或 -∞

+∞ ⇒ 0x 7F80 0000

-∞ ⇒ 0x FF80 0000

下溢出处理：将运算结果按机器零存储

Underflow Exception 置 1

IEEE 默认不响应下溢出，程序不中断

+0 ⇒ 0x0000 0000

-0 ⇒ 0x8000 0000

非规格化浮点数最小正数 2^{-149}

最大正数 $2^{-126}-2^{-149}$

非规格化 ⊆ 下溢.

e.g. 8020 0000H float 表示的 x 为

⇓

1000 0000 0010 0…0 (20个0)

1.00000000,010…0 (21个0)

尾数不全为0，$-0.01\times 2^{-126}=-2^{-128}$

NaN 产生例子.

0.0f / 0.0f sqrt(-1.0f)

∞ − ∞

数据的存储和排列

§ 大小端模式.

01 23 45 67 H　　19088743D

MSB 最高有效字节　LSB 最低有效字节

多字节数据在内存里一定是连续的n个字节.

	0800H	0801H	0802H	0803H	
大端 …	01H	23H	45H	67H	…
小端 …	67H	45H	23H	01H	…

↳ 便于机器处理.

§ 边界对齐

现代计算机按字节编址(8bit)

字节1	字节2	字节3	填充
半字		填充	
字			

* 1.0f / 0.0f == +∞

-1.0f / 0.0f == -∞

§ 加/减计算

① 0操作.

② 比阶码，朝大的对阶.

③ 尾数加/减.

④ 结果规格化

⑤ 舍入、溢出处理.

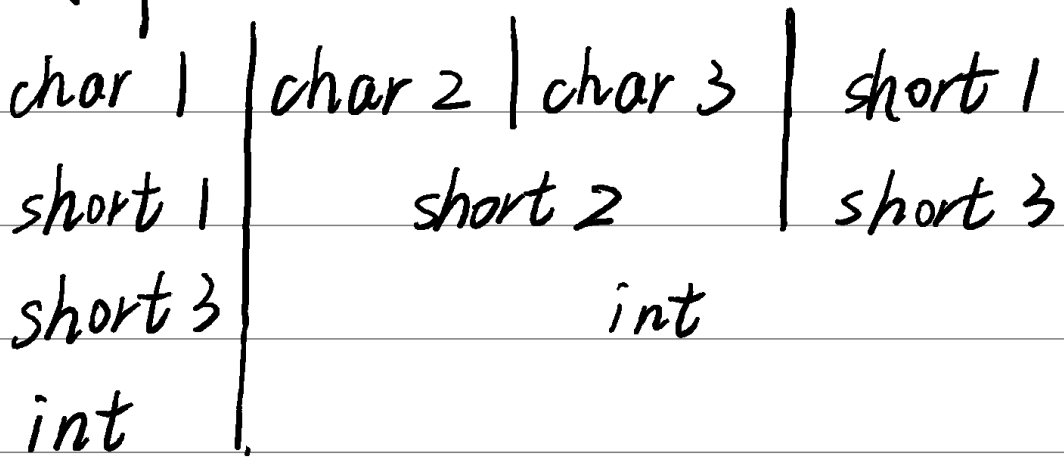
§ 大数吃小数

浮点数精度有限

		(x+y)+z	x+(y+z)
x	-1.50E+38		-1.50E+38
y	1.50E+38	0.00E+00	
z	1.0	1.0	1.50E+38
		1.00E+00	0.00E+00

- 每个字节对应1个地址
- 通常也支持按字/半字/字节寻址
- 每次访存只能读写1个字.

边界不对齐



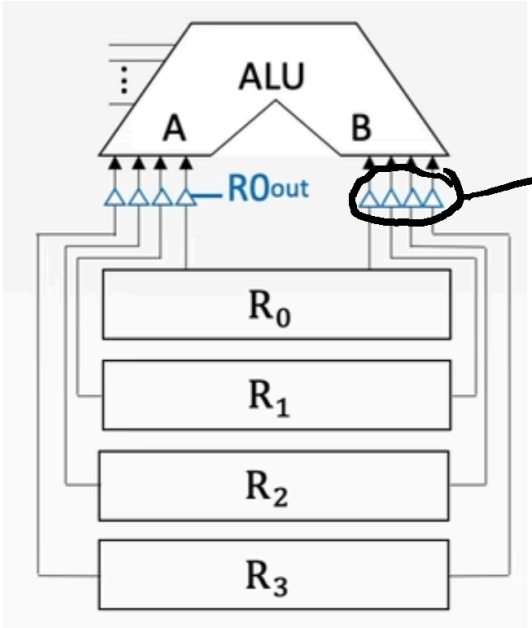
short 1 需2次访存.

CPU 功能与结构

功能：指令控制、操作控制、时间控制、数据加工、中断处理.

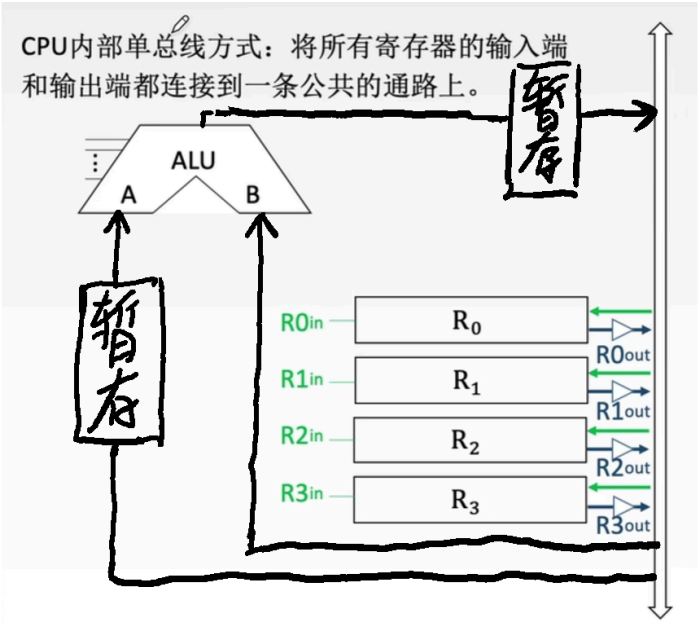
5 运算器.

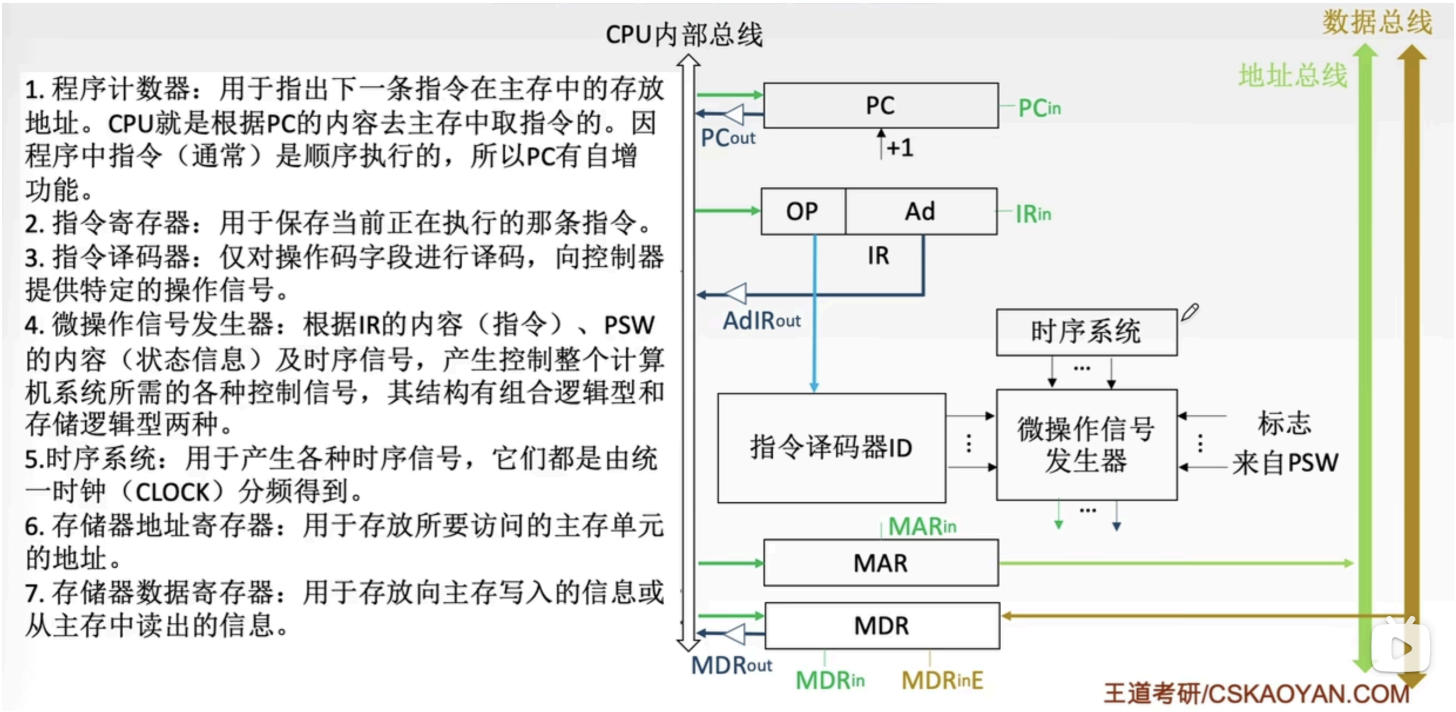
①专用数据通路：寄存器所有位置与 ALU 输入端相连.



三态门可更换为多路选择器.

②单总线（结构简单、容易实现，存在冲突、性能低）





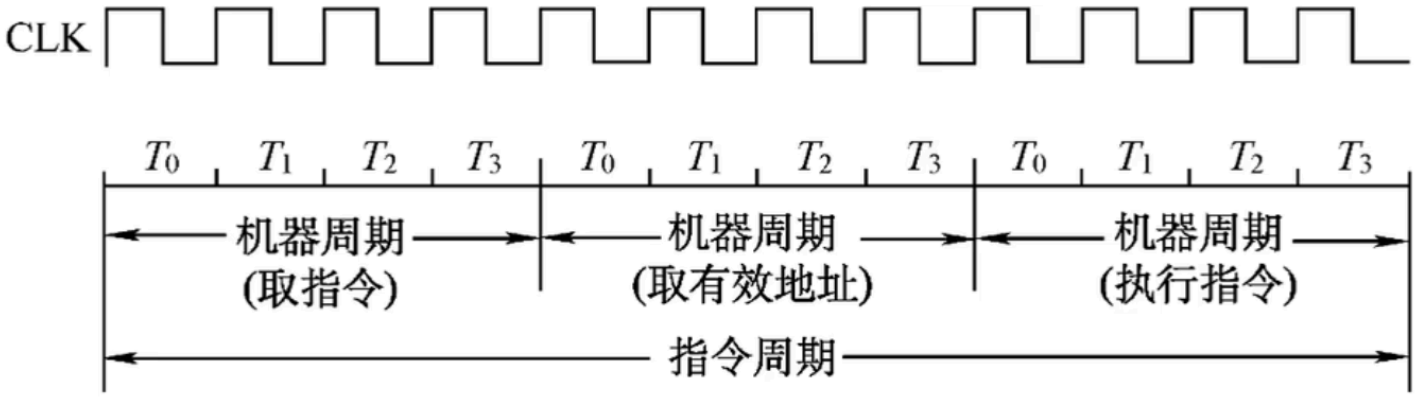
存储地址寄存器 MAR

数据 MDR

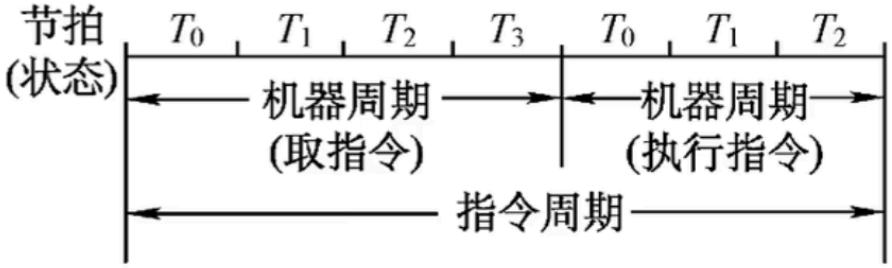
5 单总线指令执行过程

指令周期：取指（含译码）+ 执行

（取下条，PC+1）



(a) 定长的机器周期



(b) 不定长的机器周期

1. 取指.

① PC 地址 → MAR.

② CU 发出“读信号” → 主存.

③ MAR 所指内容 → MDR

④ MDR 内容转存入 IR.

⑤ PC + 1

2. 间址

① IR 地址码 → MAR

或 MDR 地址码 → MAR

② CU 控制主存读

③ MAR 所指内容 → MDR.

④* (MDR) → Ad(IR)

3. 执行

4. 中断

用堆栈保存断点（先修改指针，后存入数据） SP 指向栈顶

① SP-1 (SP) → MAR. PC 存入主存

② CU 控制主存写

③ (PC) → MDR, PC 内容入 MDR.

④ PC 修改为中断处理第1条指令向量地址

5. 指令执行方案.

① 单指令周期.

把指令周期统一为最长.

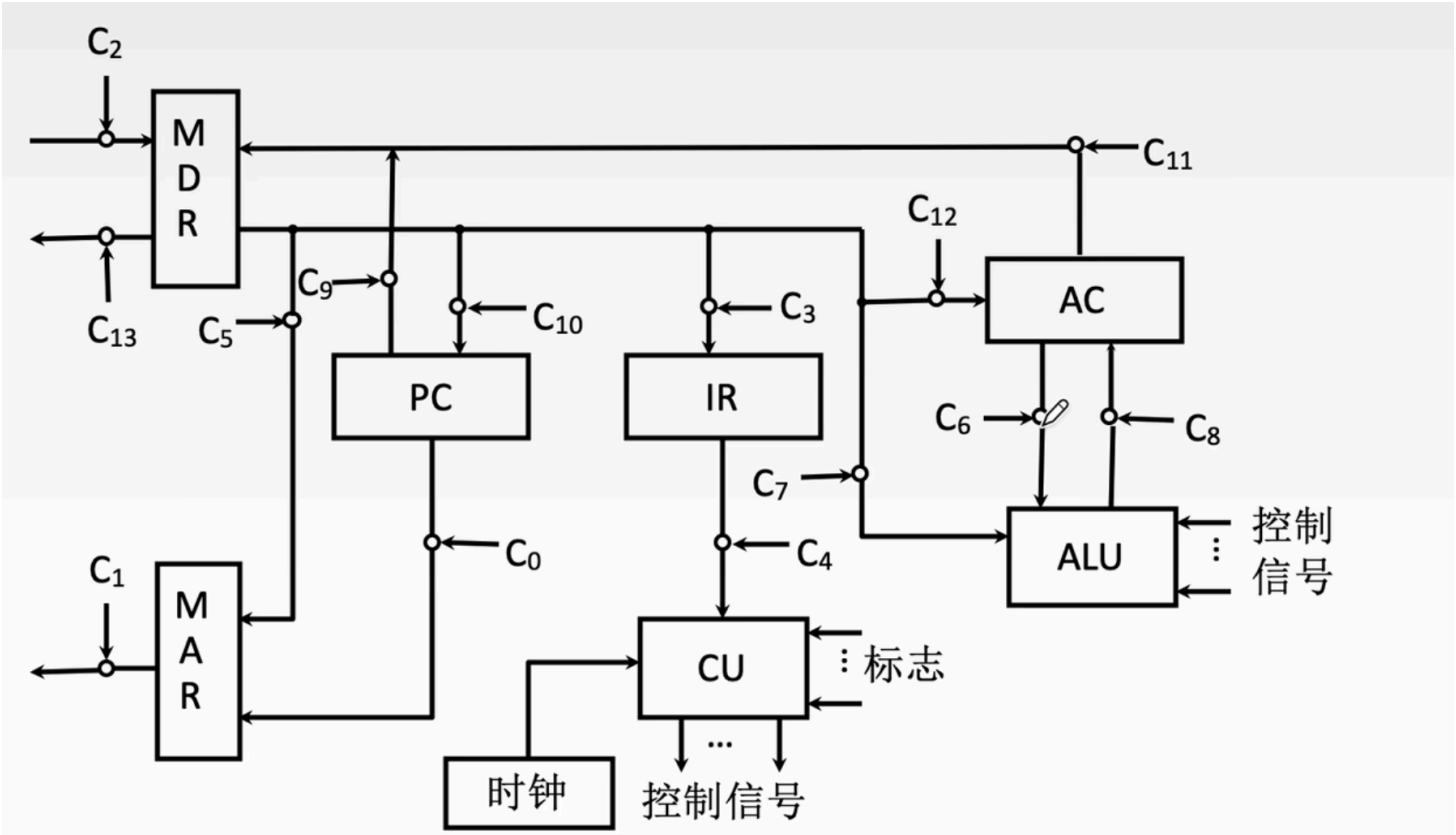
只能串行执行.

② 多指令周期

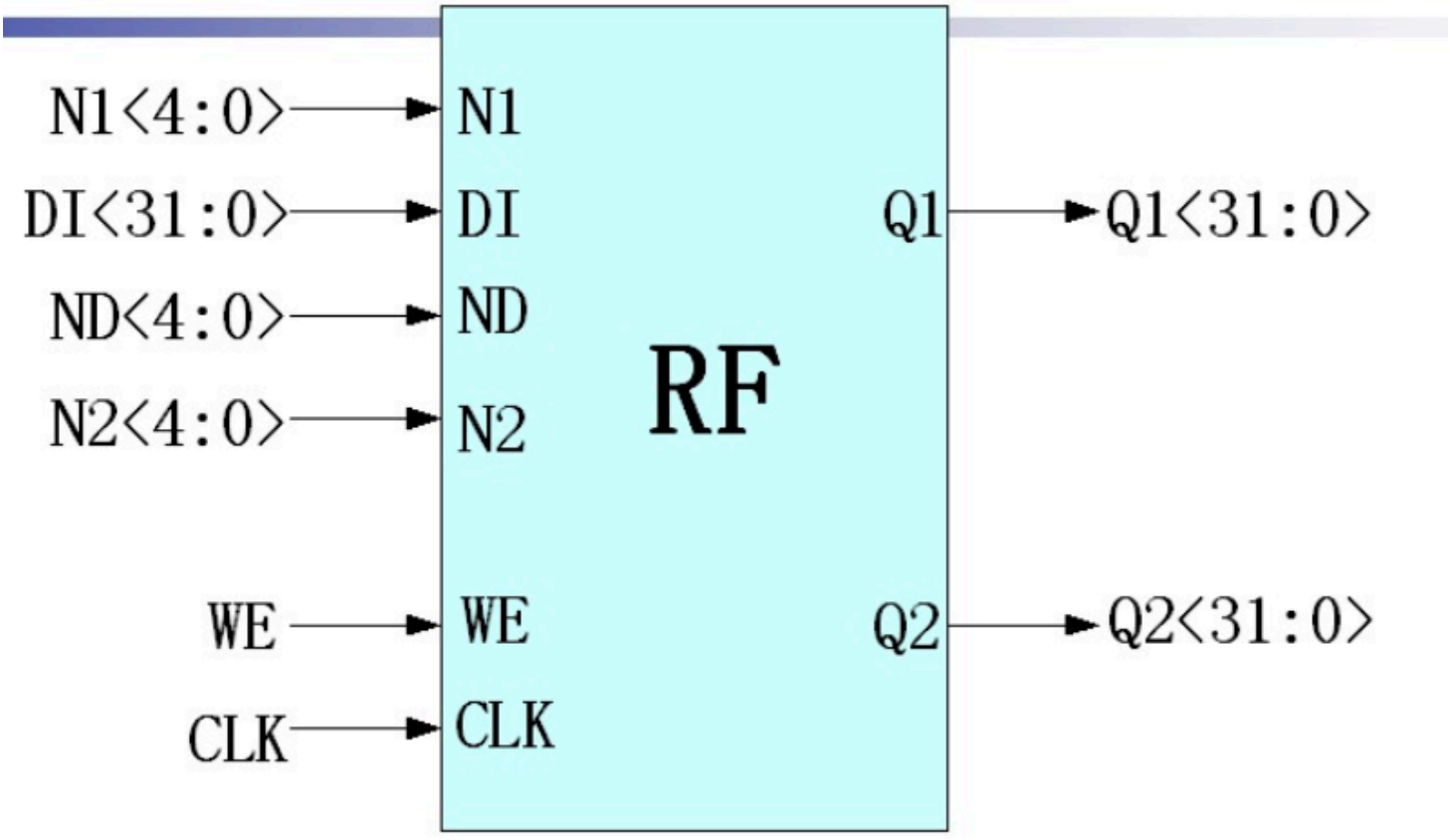
不同类型指令选用不同执行步骤来完成.

③ 流水线

6 多总线 datapath


C2
MDR
C11
C12
C13
C5
C9
C10
C3
AC
PC
IR
C6
C8
C7
ALU
控制信号
C1
MAR
C0
C4
CU
标志
时钟
控制信号

处理器.

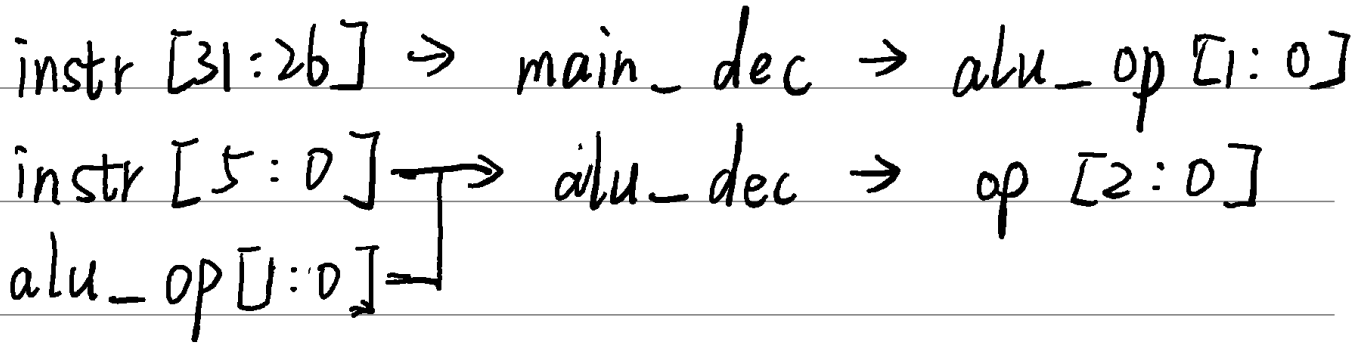

N1<4:0>
N1
DI<31:0>
DI
Q1
Q1<31:0>
ND<4:0>
ND
RF
N2<4:0>
N2
WE
WE
Q2
Q2<31:0>
CLK
CLK

1. regfile (RF)

A1, A2: 2个读端口.

AD: 写端口. DI: 写入数据.

2. ALU



op : 000 → 按位与

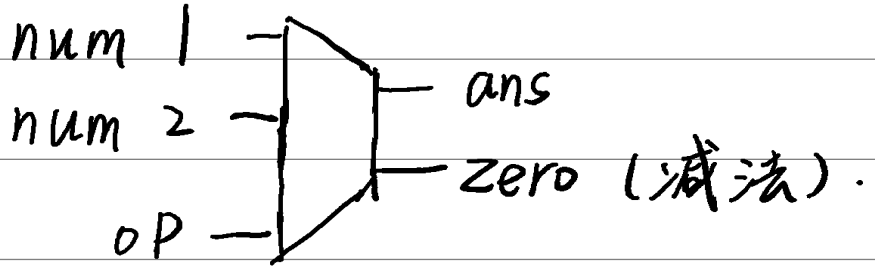
001 → 按位或

010 → 加

100 → 按位反 num 1

110 → 减

111 → 比较（<）



3. 译码与控制 controller

(main_dec + alu_dec)

(1) main_dec instr[31:26] → op.

① op = 000000 R-type

alu_op = 10

regwrite = 1 写寄存器 regdst = 1 写 rd

② op = 100011 alu_op = 00 LW

regwrite = 1 写寄存器

regdst = 0 写 rt. memtoreg = 1.

③ op = 101011 alu_op = 00 SW

memwrite = 1 写主存

memtoreg = 0 数据来源ALU计算的地址.

④ op = 000010 alu_op = 00 j

jump = 1.

⑤ op = 001000 alu_op = 00 addi

写 rt. (类似 lw)

⑥ op = 000100 alu_op = 01 beq

branch 分支 = 1 需要减法

(2) alu_dec funct ⇒ instr[5:0]

① alu_op = 00, SW用 add

② alu_op = 01, beq 和 sub.

③ alu_op = 10, R-type

funct = 100000 → add

100010 → sub

100100 → and

100101 → or

101010 → slt.

指令系统和指令格式						
31 26	25 21	20 16	15 5	4 0	指令助记符	意 义
00 0000	rd	rs1		rs2	and rd, rs1, rs2	rd ←(rs1) and (rs2)
00 0001	rd	rs1	imme		andi rd, rs1, imme	rd←(rs1) and imme
00 0010	rd	rs1		rs2	or rd, rs1, rs2	rd ←(rs1) or (rs2)
00 0011	rd	rs1	imme		ori rd, rs1, imme	rd ←(rs1) or imme
00 0100	rd	rs1		rs2	add rd, rs1, rs2	rd ←(rs1) add (rs2)
00 0101	rd	rs1	imme		addi rd, rs1, imme	rd ←(rs1) add imme
00 0110	rd	rs1		rs2	sub rd, rs1, rs2	rd ←(rs1) sub (rs2)
00 0111	rd	rs1	imme		subi rd, rs1, imme	rd ←(rs1) sub imme
00 1000	rd	rs1	imme		load rd, rs1, imme	rd ←((rs1) + imme)
00 1001	rd	rs1	imme		store rd, rs1, imme	(rd)→((rs1) + imme)
00 1010	disp				bne disp	If z=0, pc←(pc)+disp
00 1011	disp				bnq disp	If z=1, pc←(pc)+disp
00 1100	disp				branch disp	pc←(pc) + disp

单周期 CPU

§ R-type 数据通路流程

① 取指 PC → IR

② 读 reg : rs 、rt

③ ALU 执行运算

④ 写回寄存器

⑤ 更新 PC

regdst 、regwrite ⇒ 1

ALUop = 10.

§ Branch 数据通路流程

① 取指

② 读 rs 、rt

③ ALU 减法比较 (beq)

④ 分支地址计算.

[Beq]

(i) imm 立即数扩展为 32 位

(ii) 再左移 2 位 (地址偏移)

(iii) 与 (PC+4) 相加. ⇒ PC_branch

[J]

(PC+4)高4位 © 26位target、2'b00 Instr[25:0]

⑤ PC 选择更新

若 branch & zero == 1

则 PC = PC_branch

若 Jump == 1, 则 PC = PC_jump.

else PC = PC + 4

§ lw 数据通路

① PC 取指.

② 读 rs 基地址.

③ offset 扩展至32位，基地址与32位offset在ALU中相加

④ ALU_rslt 作为地址从内存读取 data 写回 rt

⑤ 更新 PC.

Memtoreg、Regwrite、MemRead => 1

ALUop = 00，地址作加法

§ sw 数据通路.

① PC 取指.

② rs 读地址

rt 读出数据.

③ ALU 计算有效地址

④ 在ALU_result 写入 rt 的数据.

⑤更新PC

*多周期CPU

①每条指令周期数不同

②时钟周期短，由(ALU计算)决定

③部件在不同周期复用.

④需要IR、MDR、MAR、ALUout暂存中间结果.

⑤由FSM控制

五级阶段：IF、ID、EX、MEM、WB.(取指、译码、执行、访存、写回)

五级流水线CPU

undefinedundefined

undefined

undefined

undefined

undefined